

宁波大学
NINGBO UNIVERSITY

网络集成实践

实验设计报告

姓名：	梁航锋
学号：	216001912
课程名称：	网络集成实习
所在学院：	信息科学与工程学院
专业班级：	21 级计算机网络工程方向
任课教师：	陈海明

实验名称	网络系统服务-Web 服务创建

一、 实验的目的与要求：

用 C++ Socket 编程实现一个 Web 服务器（端口号为 8080 或者其它）

实验要求：

1. 该 Web 服务器在一台主机上运行，支持多台主机同时访问（多线程）
2. 有服务开启和关闭的开关
3. 能够返回指定主页（index.html），页面可以是静态的内容，也可以包括实现 HTTP 的四个操作的动态交互

测试方法：

在同组其他主机上，打开浏览器，输入 http://主机 IP 地址：8080 或指定的端口号，浏览器上显示主页

完成实验报告，要求给出实现的方法的具体说明和测试结果

二、 实验的设备及软件：

2.1 实验设备：

1. 计算机 若干

2.2 实验软件：

1. C++开发环境
2. 浏览器
3. Postman 等工具

三、 系统设计的原理和流程图：

3.1 实验原理分析

本实验需要通过 C++ Socket 编程实现一个简单的 Web 服务器，使得该服务器能够处理来自多个客户端的并发请求，并返回指定的页面。服务器方面，需要通过 socket 编程与客户端进行通信，并采用多线程技术处理可能出现的多个并发请求。

对于 socket 编程，涉及的相关函数来自于 winsock2.h 头文件，服务器方面，包含创建套接字、绑定套接字、监听套接字等步骤来实现接收客户端的请求。

对于多线程处理，为了能够处理多个客户端的请求，每个客户端独立地通过一个套接字连接到 Web 服务器。

对于服务开关，最终的服务将编译为可执行文件，通过运行可执行文件以实现在指定的端口开放服务器服务，关闭相应进程以实现关闭服务。

对于 HTTP 的处理，服务器方面需要接收 HTTP 请求，并生成 HTTP 响应，同时返回客户端请求的资源，例如 index.html。

3.2 实验流程图

综上可得，实验代码的大致包括以下若干模块。

1. 服务器的初始化：套接字创建、将套接字绑定到指定的 IP 与端口、监听套接字以等待客户端连接请求。
2. 接受客户端连接：有客户端请求连接时接受连接、创建新的套接字用于客户端通信。
3. 处理客户端请求：读取 HTTP 请求报文、生成 HTTP 响应报文、根据请求内容返回请求资源
4. 接受客户端数据：接受客户端的 POST 报文，读取数据并进行处理。

实验代码的大致流程图如图 3.1 所示。

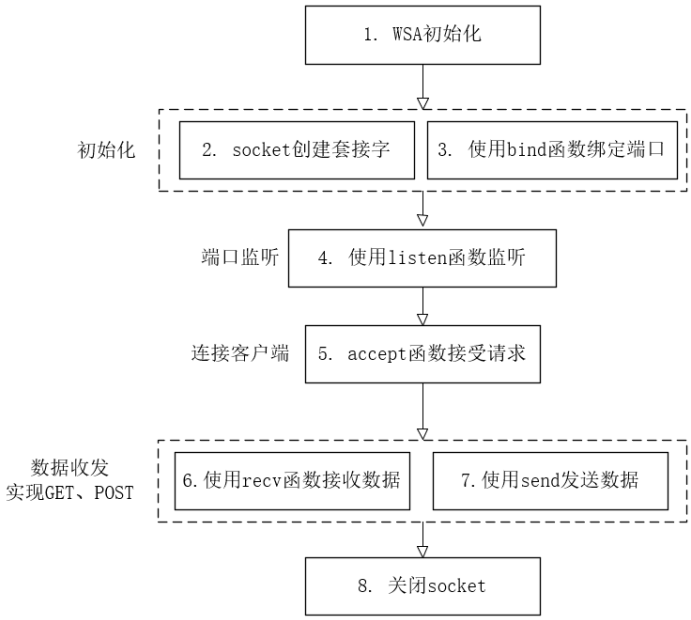


图 3.1 实验代码的大致流程图

四、 系统实现的过程、步骤及测试结果

4.1 代码设计

按照图 3.1 所示步骤依次进行代码编写，采用模块化形式进行代码的编写。

（1）辅助函数与服务初始化

为了便于检查程序的运行状态，需要对必要的错误和正确运行的情况进行说明与处理，因此编写两个信息显示函数 error 和 success，传入待输出的信息 msg，类型为 std::string，对于 error 函数，需要打印显示来自于 WSA 服务的错误类型，使用函数为 WSAGetLastError()，最后中止程序。代码如下：

```
void error(std::string msg) {
    // 输出错误信息以供检查
    std::cout << "error:" << msg << " " << WSAGetLastError() << "\n";
    exit(-1);
    return;
}
void success(std::string msg) {
    std::cout << "success:" << msg << "\n";
}
```

初始化部分，即启用 WSA 服务，根据官方文档，调用函数 WSAStartup，传入版本信息 version，需要用 WORD 字类型进行封装，具体代码如下：

```
void winsocket_init() {
    // 初始化 WSA
    WORD version = MAKEWORD(2, 2);
    WSADATA data;
    int err = WSAStartup(version, &data);
    if (err != 0) {
```

```
        error("winsocket");
    } else {
        success("winsocket");
    }
}
```

（2）服务器套接字创建

编写函数 socket_create()，用于返回一个服务器套接字，该套接字已绑定端口并正在进行访问或请求的监听。

首先使用 socket 函数进行套接字的创建，传入参数依次为：地址族（IP 类型）、数据传输方式、传输协议。在本次实验的应用场景中，设置三个参数为 AF_INET（即 IPv4）、SOCK_STREAM（数据流传输）、IPPROTO_TCP（TCP 协议）。若创建失败，返回的 socket 的值为 INVALID_SOCKET，需要报错并终止程序。

接下来要在创建的套接字上进行端口等信息的绑定，首先使用 sockaddr_in 结构体用来描述需要绑定的信息，其中包含 sin_family、sin_port、sin_addr 等成员，分别表示地址族、端口、IP 地址，具体设置为 AF_INET、htons(9999)、INADDR_ANY，含义分别为：地址类型为 IPv4，监听端口 9999（需要使用 htons()函数来将其转换为网络字节流），并且接受任意 IP 地址的访问。

使用 bind 函数进行绑定，传入待绑定套接字、sockaddr 指针、绑定信息的长度，若绑定失败则返回一个非 0 值。

最后使用 listen()函数激活监听，传入参数为套接字和能够接受的最大等待连接数目。最后将处理好的 socket 进行返回即可。

根据以上步骤编写代码，具体如下：

```
SOCKET socket_create() {
    // socket 创建
    SOCKET soc = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (soc == INVALID_SOCKET) {
        error("socket");
        return 1;
    } else {
        success("socket create");
    }
    // ip 绑定
    // sockaddr 结构体中存储协议、端口、IP 地址等信息
    // 与 socket 绑定
    struct sockaddr_in addrWebServer;
    addrWebServer.sin_family = AF_INET;
    addrWebServer.sin_port = htons(9999);
    addrWebServer.sin_addr.S_un.S_addr = INADDR_ANY;
    // 调用 bind 函数进行绑定
    int err = bind(soc, (struct sockaddr*) &addrWebServer,
sizeof(addrWebServer));
    // 处理可能的报错信息
```

```
    if (err != 0) {
        closesocket(soc);
        error("socket bind");
        return 1;
    } else {
        success("socket bind");
    }
    // 监听端口，第二个参数为最大等待连接数目
    listen(soc, 20);
    // 返回套接字
    return soc;
}
```

（3）HTTP 请求解析与主页发送

在完成服务器套接字的创建后，调用 accept 函数可以在有客户端连接的时候进行连接的接受（三次握手），能够得到连接套接字，通过该套接字，调用 recv 函数可以接受来自客户端的消息，通过 send 函数可以进行消息的发送。

除此之外，也可能接收到 POST 类型的请求，而通过 recv 函数接收到的消息为一系列字符数组，为方便后续对请求的处理，需要对请求信息进行解析。编写结构体 HttpRequest 用于描述一个 HTTP 请求，其中包含请求方式、URL、版本、头部，即描述一个完整的 HTTP 请求。编写函数 parseRequest()，传入待解析的消息，采用字符流 istringstream 进行逐行处理，具体代码如下：

```
// 简易表示一个HTTP 请求
struct HttpRequest {
    // 请求方式
    std::string method;
    // 请求来源URL
    std::string url;
    // 请求版本
    std::string version;
    // 其它头部信息
    std::unordered_map<std::string, std::string> headers;
    // 打印信息
    void print_info() {
        std::cout << "Method: " << method << std::endl;
        std::cout << "URL: " << url << std::endl;
        std::cout << "Version: " << version << std::endl;
        std::cout << "Headers:" << std::endl;
        for (const auto& header : headers) {
            std::cout<<header.first<<":" << header.second << std::endl;
        }
    }
};
// 解析请求消息，封装为结构体
```

```
HttpRequest parseRequest(const std::string& request) {
    HttpRequest httpRequest;
    std::istringstream requestStream(request);
    std::string line;
    bool isFirstLine = true;
    while (std::getline(requestStream, line)) {
        // 移除 \r
        if (!line.empty() && line.back() == '\r') {
            line.pop_back();
        }
        if (isFirstLine) {
            // istringstream 解析第一行获取请求方式、URL、版本
            std::istringstream lineStream(line);
            lineStream >> httpRequest.method >> httpRequest.url >> http
Request.version;
            isFirstLine = false;
        } else {
            // 解析头部信息
            size_t colonPos = line.find(':');
            if (colonPos != std::string::npos) {
                std::string headerName = line.substr(0, colonPos);
                std::string headerValue = line.substr(colonPos + 1);
                // 移除前导空格
               headerValue.erase(0,headerValue.find_first_not_of(" "));
               httpRequest.headers[headerName] = headerValue;
            }
        }
    }
    return httpRequest;
}
```

对于初次请求，接收到的是 HTTP 请求报文，需要处理并返回 HTTP 响应报文，内容包括响应标识符 200、服务器名称、发送的数据类型等报文头部信息，同时要注意设置发送数据类型 Content-Type 为 text/html;。

而对于主页的发送，即发送 index.html 文件，使用 fopen 函数打开文件，使用 fgets 函数逐行获取文件数据，使用 feof 判断是否读到文件尾了。具体的实现代码如下：

```
void accept_send(SOCKET soc) {
    // 接受连接后发送消息
    char recvBuf[1024] = "";
    int err = recv(soc, recvBuf, sizeof(recvBuf), 0);
    if (err <= 0) {
        error("receive");
    }
```

```
    // 解析接收到的数据
    HttpRequest httpRequest = parseRequest(recvBuf);
    // 打印显示
    httpRequest.print_info();
    if (httpRequest.method == "GET") {
        // 发送HTTP 响应，包括协议、服务器名称、内容长度等报文头部消息
        char protocol[] = "HTTP/1.0 200 OK\r\n";
        char serverName[] = "server:lhf web server\r\n";
        char cntLen[] = "Content-Length:2048\r\n";
        // 内容类型设置为text/html，初始发送的是首页作为响应
        char cntType[]="Content-Type:text/html; charset=utf-8\r\n\r\n";
        // 调用send 函数以发送数据
        send(soc, protocol, strlen(protocol), 0);
        send(soc, serverName, strlen(serverName), 0);
        send(soc, cntLen, strlen(cntLen), 0);
        send(soc, cntType, strlen(cntType), 0);
        // 打开HTML 文件
        FILE* ps = fopen("index.html", "r");
        if (ps == NULL) {
            error("FileOpen");
        }
        // 循环从文件标识器中读取数据并调用send 函数发送。
        char data[1024] = "";
        do {
            fgets(data, 1024, ps);
            send(soc, data, strlen(data), 0);
        } while (!feof(ps));
        fclose(ps);
    } else if(httpRequest.method == "POST") {
        // ...POST 请求的处理
    }
    return ;
}
```

（4）对于 POST 请求的处理

为测试 POST 请求的处理功能，编写简单的处理函数，用于接收来自于客户端的两个数字，并将这两个数字进行相乘后返回客户端。在上一小部分的代码空余处进行填写，内容如下，使用 pair 数对接收待处理的两个数字，函数 parse_numbers()中从请求消息中获取两个数字，经过处理后编写响应报文，并把处理后的数字进行返回。

```
std::pair<int,int> numbers = parse_numbers(recvBuf);
int sum = numbers.first * numbers.second;
std::string response = "HTTP/1.1 200 OK\r\nContent-Type: text/plain\r\n
```

```
\r\n" + std::to_string(sum);
send(soc, response.c_str(), response.size(), 0);
```

（5）主函数内容

完成上述函数内容的编写后，在主函数中进行调用和处理，对于等待接收部分，使用 while(true)进行持续监听，并在 accept 函数处等待，当有客户端发送请求后，accept 函数处理得到套接字，并进行后续的消息接收、解析、处理操作。为了实现并发控制，以处理多个可能的连接，使用 thread 和 detach 在后台为连接运行新的线程，线程处理函数中包含前面编写好的 accept_send 函数。具体代码如下：

```
void client_handler(SOCKET clientSoc) {
    std::cout << "Handling new client in thread\n";
    accept_send(clientSoc);
}
int main(int argc, char const *argv[]) {
    // 初始化 socket 服务（WSA）
    winsocket_init();
    // 初始化服务器套接字
    SOCKET serverSoc = socket_create();
    struct sockaddr_in addrClient;
    int length_addrClient = sizeof(addrClient);
    while (true) {
        // 接受一条 TCP 连接，使用 accept 函数，返回连接的套接字
        SOCKET clientSoc = accept(serverSoc, (struct sockaddr*)&addrCli
ent, &length_addrClient);
        if (clientSoc == INVALID_SOCKET) { error("accept"); }
        std:: cout << "connected..\n";
        // 接受连接后，接收消息并解析处理
        std::thread(client_handler, clientSoc).detach();
         // 关闭客户端套接字
         closesocket(clientSoc);
    }
    // 关闭服务器套接字
    closesocket(serverSoc);
    // 关闭 WSA 服务
    WSACleanup();
    return 0;
}
```

4.2 系统测试

运行经过编译后的可执行文件，观察控制台输出，如图 4.1 所示，可以看到显示 success 提示，表示 WSA 服务启用、套接字创建、端口绑定等操作顺利完

从。此时服务器等待客户端的连接。

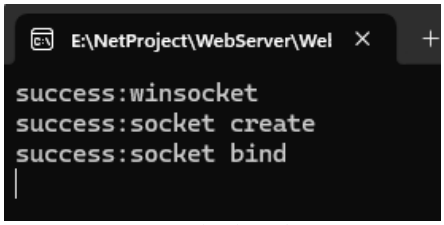


图 4.1 初始化过程

为方便测试，使用 postman 进行请求的发送，尝试发送 GET 类型的请求，观察控制台输出，注意服务开放在端口 9999，需要使用 IP:9999 的方式进行访问，控制台输出如图 4.2 所示。可以看到请求类型、URL、版本、报文头部信息均被顺利解析。

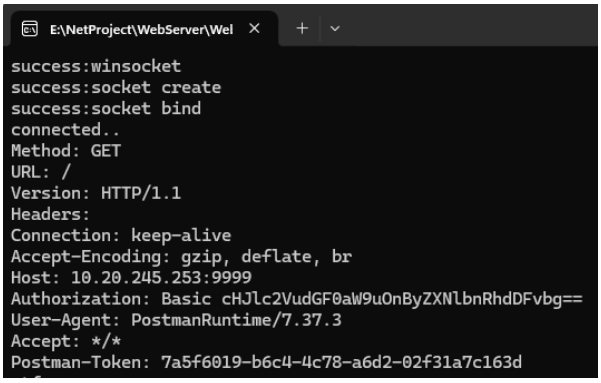


图 4.2 GET 请求的报文信息

编辑简易的 HTML 文件，放置于与代码源文件同一路径下，HTML 代码内容如下图 4.3 所示。

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>两数相乘</title>
    <script>
        function sendRequest() {
            var num1 = document.getElementById("num1").value;
            var num2 = document.getElementById("num2").value;
            var xhr = new XMLHttpRequest();
            var url = "http://10.20.245.253:9999";
            var params = "num1=" + num1 + "&num2=" + num2;

            xhr.open("POST", url, true);
            xhr.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
            xhr.send(params);
            xhr.onreadystatechange = function() {
                if (xhr.readyState == 4 && xhr.status == 200) {
                    document.getElementById("result").innerText = "Result: " + xhr.responseText;
                }
            };

        }
    </script>
</head>
<body>
    <h1>两数相乘</h1>
    <form onsubmit="sendRequest(); return false;">
        <label for="num1">数字 1:</label>
        <input type="text" id="num1" name="num1"><br><br>
        <label for="num2">数字 2:</label>
        <input type="text" id="num2" name="num2"><br><br>
        <input type="submit" value="发送给服务器处理">
    </form>
    <p id="result"></p>
</body>
</html>
```

图 4.3 HTML 代码内容

使用同一子网下的主机访问运行服务器的主机 IP，需要添加服务端口 9999，结果如图 4.4 所示。可以看到成功渲染界面，可以说明 GET 请求的响应成功，

能够将主页文件发送到客户端，同时两数的相乘结果也正确，由此判断 POST 请求得到了正确的处理。

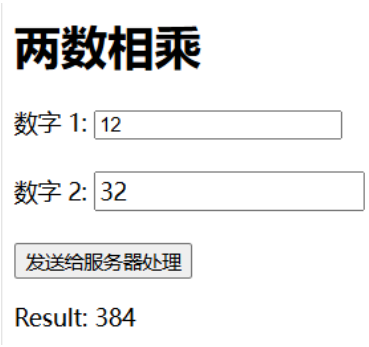


图 4.4 POST 效果

使用 Postman 进行验证，结果如下图所示，使用 Post 进行请求，附带上两数的参数，这与 HTML 中的操作一致，结果显示返回得到相乘的数，符合实验预期。使用多台主机以及 Postman 进行同步验证，结果均能够顺利访问，由此验证并发访问的处理正确性。

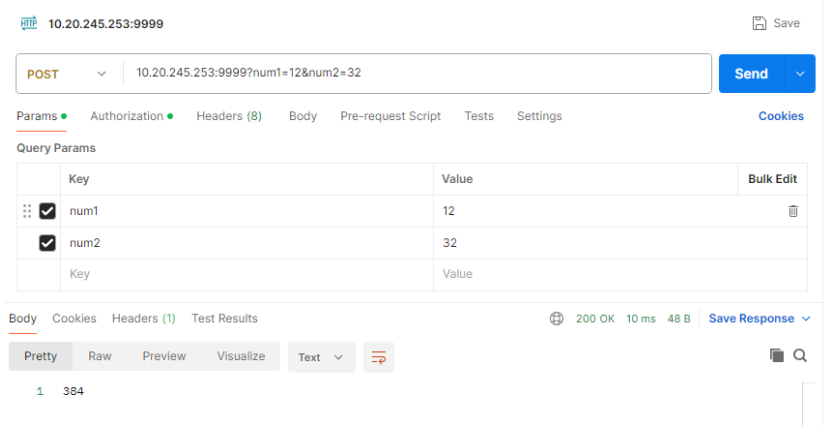


图 4.4 Postman 验证 POST 效果

五、 教师批阅意见：

成绩评定：

教师签字：

年 月 日

六、备注：